MS PACMAN

Empezamos el desarrollo del controlador de MsPacMan con la idea de usar nodos para ponerle un peso a las posiciones que se elijan en las intersecciones.
- **MsPacManNodosIniciales, que contiene un HashMap con <Integer, Double> con su posición y costo.**

Después de adaptar esta metodología queríamos que los fantasmas también tuvieran su influencia sobre los nodos, por lo que le pedimos a “ChatGPT” que nos plantease algo para poder abordar esta metodología. Sacando este método que adaptamos para que se entendiese mejor, además de modificar repintarCostes para que quedase más claro la construcción de nodos con sus costes:

Búsqueda propia restringida	Una anchura sobre el grafo que sólo cruza nodos seguros contesta, en una pasada, cuál es el objetivo alcanzable más cercano *y* por dónde se va. `getClosestNodeIndexFromNodeIndex` no sabe que hay fantasmas.

```
private void añadirPeligroFantasmas(Game game) {

    for (GHOST fantasma : GHOST.values()) {
        // Si está en la guarida, no supone peligro
        if (game.getGhostLairTime(fantasma) > 0) continue;
        int nodoFantasma = game.getGhostCurrentNodeIndex(fantasma);

        if (nodoFantasma == -1) continue;
        if (game.isGhostEdible(fantasma)) añadirApetitoFantasmas(game, fantasma);

         for (Map.Entry<Integer, Double> entrada : costes.entrySet()) {
            int nodo = entrada.getKey();
            int distancia = game.getShortestPathDistance(nodo, nodoFantasma);
            if (distancia < 0) continue;
            double peligro = 0.0;   //  Cuanto más cerca esté el fantasma,
                                    //  mayor será la penalización.
            if (distancia == 0) {//  distancia 0 -> peligro máximo
                peligro = PELIGRO_BASE * 4;
            }
            else if (distancia == 1) {//  distancia 1 -> peligro alto
                peligro = PELIGRO_BASE * 2;
            }
            else if (distancia == 2) {//  distancia 2 -> peligro medio
                peligro = PELIGRO_BASE;
            }
            else if (distancia <= CERCA) {
                peligro = 100.0 / distancia; // Peligro decreciente con la distancia
            }
            entrada.setValue(entrada.getValue() + peligro);
        }
    }
}
```

- **MsPacManFantasmasPeligro, que añade valores a los nodos en los que se posicionan los fantasmas. Si alguno es comestible se cambiará el peligro a apetito para que pueda tener en cuenta que se los puede comer.**

- **MsPacManEntrega, para la entrega quisimos adaptar estas dos funcionalidades,**

Mirar dos cruces por delante	Un tramo seguro que desemboca en un cruce sin salidas seguras es una trampa. Con profundidad 2 se detecta; con más, la predicción ya no vale nada.

«¿Llego yo antes?»	Comparar tu distancia a un cruce con la de cada fantasma *a ese mismo cruce*, con un margen. Es la diferencia entre huir y ponerse a salvo.

Para ello, volvimos a pedir ayuda a la IA cómo podríamos abordar este problema, por lo que nos recomendó refactorizar el map a varias listas con sus costes, y luego esos costes se combinarán en una lista principal para poder hacer las multiplicaciones más claramente, por lo que creamos [dist (píldora más cercana), peligro (acumulado de los fantasmas), apetito (por los fantasmas comestibles), callejón (que añade seguridad cuantas más salidas tenga), grado (número de vecinos del nodo) y coste (el combinado que se usa para consultar la mejor opción)].

GHOST

Adaptamos esta pieza de código de la wiki de la asignatura

```
// Los papeles, en código: un switch y cuatro estrategias distintas
private MOVE segunSuPapel(Game game, GHOST f) {
    int  nodo = game.getGhostCurrentNodeIndex(f);
    MOVE ult  = game.getGhostLastMoveMade(f);
    int  pacman = game.getPacmanCurrentNodeIndex();

    switch (f) {
        case BLINKY:   // persigue por el camino más corto
            return primerMovimientoDe(game, game.getShortestPath(nodo, pacman, ult));

        case PINKY:    // intercepta: va al próximo cruce de Ms. Pac-Man
            int destino = proximoCruceDe(game, pacman, game.getPacmanLastMoveMade());
            return primerMovimientoDe(game, game.getShortestPath(nodo, destino, ult));

        case INKY:     // cubre la píldora de poder activa más cercana a Ms. Pac-Man
            int[] pp = game.getActivePowerPillsIndices();
            if (pp.length > 0) {
                int guardar = game.getClosestNodeIndexFromNodeIndex(pacman, pp, DM.PATH);
                return primerMovimientoDe(game, game.getShortestPath(nodo, guardar, ult));
            }
            return primerMovimientoDe(game, game.getShortestPath(nodo, pacman, ult));

        default:      // SUE: persigue, pero de vez en cuando se sale del guion
            return (game.getCurrentLevelTime() % 10 == 0)
                    ? movimientoAleatorioLegal(game, nodo, ult)
                    : primerMovimientoDe(game, game.getShortestPath(nodo, pacman, ult));
    }
}

private MOVE primerMovimientoDe(Game game, int[] camino) {
    if (camino == null || camino.length < 2) return MOVE.NEUTRAL;
    return game.getMoveToMakeToReachDirectNeighbour(camino[0], camino[1]);
}
```

- **GhostsConComportamientoInicial** (La técnica de papeles distintos, el esqueleto utilizado de la wiki con el switch y cuatro estrategias distintas)

- **GhostsConDispersiónEntrega**

Este último trata de que si más de dos fantasmas están demasiados juntos persiguiendo a Ms pacman, hacemos que se dispersen.

```java
private boolean necesitanDispersarse(Game game) {

    int activos = 0;

    for (GHOST f : GHOST.values()) {

        if (game.getGhostLairTime(f) > 0) continue;

        int nodo = game.getGhostCurrentNodeIndex(f);

        if (nodo == -1) continue;

        activos++;
    }

    // Con menos de 2 no tiene sentido dispersarse
    if (activos < 2) return false;

    for (GHOST f1 : GHOST.values()) {

        if (game.getGhostLairTime(f1) > 0)continue;

        int n1 = game.getGhostCurrentNodeIndex(f1);

        if (n1 == -1) continue;

        for (GHOST f2 : GHOST.values()) {

            if (f1 == f2) continue;

            if (game.getGhostLairTime(f2) > 0) continue;

            int n2 = game.getGhostCurrentNodeIndex(f2);

            if (n2 == -1) continue;

            int distancia = game.getShortestPathDistance(n1, n2);

            if (distancia >= 0 && distancia < DISTANCIA_MINIMA_FANTASMAS) return true;
        }
    }

    return false;
```

Para ello, hemos aplicado un peso a esta dispersión para que cuando más esté alejado de los otros fantasmas, mejor será el peso y si están cerca de las intersecciones, este peso también aumenta.

```java
private double puntuacionDispersión(Game game, GHOST fantasma, int candidato) {

    double puntuacion = 0.0;

    for (GHOST otro : GHOST.values()) {

        if (otro == fantasma) continue;

        if (game.getGhostLairTime(otro) > 0) continue;

        int nodoOtro = game.getGhostCurrentNodeIndex(otro);

        if (nodoOtro == -1) continue;

        int distancia = game.getShortestPathDistance(candidato,nodoOtro);

        if (distancia < 0) continue;

        puntuacion += distancia * 10.0;

        if (distancia <= 2)
            puntuacion -= 500;

        else if (distancia <= 4)
            puntuacion -= 200;

        else if (distancia <= 6)
            puntuacion -= 50;
    }

    if (game.isJunction(candidato)) {
        puntuacion += 30;
    }

    int pacman = game.getPacmanCurrentNodeIndex();

    int distanciaPacman = game.getShortestPathDistance(candidato, pacman);

    if (distanciaPacman >= 0) {
        puntuacion += distanciaPacman * 2.0;
    }

    return puntuacion;
}
```

Tomando inspiración de la técnica de papeles distintos, también hemos programado de que tengan también una preferencia de movimiento, así para evitar que todos vayan por el mismo camino de forma regular.

```java
private MOVE movimientoDispersar(Game game, GHOST fantasma) {

    int nodo = game.getGhostCurrentNodeIndex(fantasma);

    MOVE ultimo = game.getGhostLastMoveMade(fantasma);

    MOVE[] opciones = game.getPossibleMoves(nodo, ultimo);

    if (opciones == null || opciones.length == 0) return MOVE.NEUTRAL;

    MOVE mejor = opciones[0];
    double mejorPuntuacion = Double.NEGATIVE_INFINITY;

    for (MOVE m : opciones) {
        int vecino = game.getNeighbour(nodo, m);

        if (vecino == -1) continue;

        double puntuacion = puntuacionDispersión(game,fantasma,vecino);

        switch (fantasma) {

            case BLINKY:
                if (m == MOVE.LEFT) puntuacion += 40;
                break;

            case PINKY:
                if (m == MOVE.UP) puntuacion += 40;
                break;

            case INKY:
                if (m == MOVE.RIGHT) puntuacion += 40;
                break;

            case SUE:
                if (m == MOVE.DOWN) puntuacion += 40;
                break;
        }

        if (puntuacion > mejorPuntuacion) {
            mejorPuntuacion = puntuacion;
            mejor = m;
        }
    }
```

Dentro del esqueleto dado, hemos programado el “huirConCriterio” que es llamado cuando los fantasmas se vuelven vulnerables, estos se alejen de pacman pero si existe algún fantasma no vulnerable en la partida, los fantasmas que sí son vulnerables irán hacia su dirección para protegerse de Ms pacman.

```java
private MOVE huirConCriterio(Game game, int nodo, MOVE ultimo, int pacman, GHOST ghostType)

    for (GHOST f : GHOST.values()) {
        if (!game.isGhostEdible(f))
        {
            return game.getApproximateNextMoveAwayFromTarget(
                    game.getGhostCurrentNodeIndex(ghostType),
                    game.getGhostCurrentNodeIndex(f),
                    game.getGhostLastMoveMade(ghostType),
                    DM.PATH);
        }
    }

    return game.getApproximateNextMoveAwayFromTarget(
                    game.getGhostCurrentNodeIndex(ghostType),
                    game.getPacmanCurrentNodeIndex(),
                    game.getGhostLastMoveMade(ghostType),
                    DM.PATH);
        }
```